COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	August 4, 2026	**DATE SUBMITTED:**	August 11, 2026
NAME:	Benj Federic C. De Leon	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 5.1 **Queues**

1. PROCEDURE OUTPUT

ILO A: Create queue using C++ STL

Source file

```
// ILO A: Create queue using C++ STL

#include <iostream>
#include <queue>

//prototype
void display(std::queue<char> r);

int main(){

    std::queue<char> q;

    //push
    q.push('c'); // front
    q.push('p');
    q.push('e');
    q.push('0');
    q.push('1');
    q.push('0'); // rear



    //empty()
    std::cout<<"Is the queue empty: " << q.empty()<<std::endl;
```

```
    //size
    std::cout<<"Size of the queue: "<< q.size()<<std::endl;

    //front
    std::cout<< "Front of the queue: "<<q.front()<<std::endl; // front

    //back
    std::cout<<"Back of the queue: "<<q.back()<<std::endl;

    //pop
    std::cout<<"Pop: ";
    q.pop();

    //push another element
    std::cout<< "Push another element: ";
    q.push('b');

    //dislpay
    std::cout<<"Displaying element/s: ";
    display(q);

    //delete the queue
    while(!q.empty()){
        q.pop();
    }

}


void display(std::queue<char> r){
    // crete a copy of the queue
    std::queue<char> c = r;

    while(!c.empty()){
        std::cout << " "<<c.front();
        c.pop();
```

```
    }
    std::cout<<"\n";
}
```

Header source file

```
#ifndef QUEUE_H
#define QUEUE_H
#include <iostream>

template <typename T>
class qNode{
    public:
    T data;
    qNode *next;

};

//creating a new node.
template  <typename T>
qNode<T>*  new_node(T newdata){
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newdata;
    newNode->next = nullptr;
    return newNode;

}

template<typename T>
void enqueue (qNode<T>** frontPtr, qNode<T>** backPtr, T newData){
    //creating a NEW NODE
    qNode<T>* newNode = new_node(newData);

    //INSERTING TO AN EMPTY QUEUE
    if((*frontPtr) == nullptr && (*backPtr)==nullptr){
        (*frontPtr) = newNode;
```

```
        (*backPtr) = newNode;
    }
    //inserting an item into a non empt queue
    else {
        // point the backPtr NEXT TO THE newNode
        (*backPtr)->next = newNode;
        (*backPtr) = newNode;
    }
}

template<typename T>
void dequeue (qNode<T>** frontPtr, qNode<T>** backPtr){
    //check if the queue is empty to prevent crashes
    if((*frontPtr) == nullptr){
        return;
    }

    //createa temporary node to store the node to be deleted
    qNode<T>* deleteNode = nullptr;
    deleteNode = (*frontPtr);

    //check if the queue is only 1 node
    if((*frontPtr) == (*backPtr)){
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete deleteNode;
        return;
    }

    //deleting of the node
    (*frontPtr) = deleteNode->next;
    delete deleteNode;

    }

    //display all elements in the list
template<typename T>
```

```
void display (qNode<T>* frontPtr){
    qNode<T> *temp = frontPtr;

    while(temp !=nullptr){
        std::cout<< " " << temp->data;
        temp = temp->next;
    }
}

    //return the front
template<typename T>
T front(qNode<T>* frontPtr){
    return frontPtr->data;
}

#endif
```

OUTPUT

```
Is the queue empty: 0
Size of the queue: 6
Front of the queue: c
Back of the queue: 0
Pop: Push another element: Displaying element/s:  p e 0 1 0 b
```

Code analysis:

This file tests C++'s built-in std:: queue. It pushes 6 characters in, then runs through every basic queue operation, it checks if it's empty, checking size, peeking at front/back, popping one off, pushing one more, then displaying everything.

B.1. Linked List Implementation

Source file

```cpp
#include <iostream>
#include "1queuell.h"

int main(){
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    enqueue(&front, &back, 'J');
    std::cout << front->data << " " << back->data << std::endl;

    enqueue(&front, &back, 'I');
    std::cout << front->data << " " << back->data << std::endl;

    enqueue(&front, &back, 'M');
    std::cout << front->data << " " << back->data << std::endl;

    dequeue(&front, &back);
    std::cout << front->data << " " << back->data << std::endl;

    return 0;
}
```

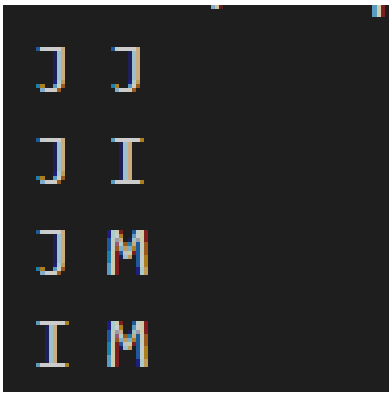
Header source file

```cpp
#ifndef QUEUE_H
#define QUEUE_H
#include <iostream>

template <typename T>
class qNode {
public:
    T data;
    qNode *next;
};

// creating a new node
```

```cpp
template <typename T>
qNode<T>* new_node(T newdata) {
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newdata;
    newNode->next = nullptr;
    return newNode;
}

template<typename T>
void enqueue(qNode<T>** frontPtr, qNode<T>** backPtr, T newData) {
    // creating a NEW NODE
    qNode<T>* newNode = new_node(newData);

    // INSERTING TO AN EMPTY QUEUE
    if((*frontPtr) == nullptr && (*backPtr) == nullptr) {
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
    }
    // inserting an item into a non empty queue
    else {
        // point the backPtr NEXT TO THE newNode
        (*backPtr)->next = newNode;
        (*backPtr) = newNode;
    }
}

template<typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr) {
    // check if the queue is empty to prevent crashes
    if((*frontPtr) == nullptr) {
        return;
    }

    // create a temporary node to store the node to be deleted
    qNode<T>* deleteNode = nullptr;
    deleteNode = (*frontPtr);
```

```
    // check if the queue is only 1 node
    if((*frontPtr) == (*backPtr)) {
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete deleteNode;
        return;
    }

    // deleting of the node
    (*frontPtr) = deleteNode->next;
    delete deleteNode;
}

// display all elements in the list
template<typename T>
void display(qNode<T>* frontPtr) {
    qNode<T> *temp = frontPtr;
    while(temp != nullptr) {
        std::cout << " " << temp->data;
        temp = temp->next;
    }
}

// return the front
template<typename T>
T front(qNode<T>* frontPtr) {
    return frontPtr->data;
}

#endif
```

OUTPUT



Code analysis:

This code builds a custom queue from scratch using linked nodes. It manually manages memory pointers to add items at the end and remove items from the front.

B.2. Array-Based Implementation

Source file

```
// PART B.2: Array-Based (Circular) Queue Implementation

#include <iostream>
#include "2queuearr.h"

int main() {
    queueArr<int> myQueue(5);   // a queue that can hold 5 numbers

    std::cout << "Enqueue 10, 20, 30, 40, 50\n";
    myQueue.Enqueue(10);
    myQueue.Enqueue(20);
    myQueue.Enqueue(30);
    myQueue.Enqueue(40);
    myQueue.Enqueue(50);

    std::cout << "\nFront: " << myQueue.Front() << "\n";
```

```
    std::cout << "Back : " << myQueue.Back() << "\n";
    std::cout << "Size : " << myQueue.Size() << "\n";
    std::cout << "Full?: " << std::boolalpha << myQueue.Full() << "\n";

    std::cout << "\nDequeue twice\n";
    myQueue.Dequeue();
    myQueue.Dequeue();
    std::cout << "Front now: " << myQueue.Front() << "\nSize: " <<
myQueue.Size() << "\n";

    std::cout << "\nEnqueue 60, 70\n";
    myQueue.Enqueue(60);
    myQueue.Enqueue(70);
    std::cout << "Back now: " << myQueue.Back() << "\nFull?: " <<
myQueue.Full() << "\n";

    std::cout << "\nCopy constructor:\n";
    queueArr<int> copy(myQueue);
    std::cout << "\ncopy.Front(): " << copy.Front() << "\ncopy.Back():
" << copy.Back() << "\n";

    std::cout << "\nClear()\n";
    myQueue.Clear();
    std::cout << "Empty? " << myQueue.Empty() << "\n";

    return 0;
}
```

Header source file

```
#ifndef QUEUEARR_H
#define QUEUEARR_H

#include <cstddef>    // gives us size_t
#include <stdexcept>  // gives us std::runtime_error
```

```cpp
template <typename T>
class queueArr {
private:
    T* data;            // the array that holds the items
    size_t capacity;     // how many items the array can hold
    size_t frontIndex;   // index of the front item
    size_t backIndex;    // index of the back item
    size_t count;        // how many items are currently stored

public:
    bool Empty();
    bool Full();
    size_t Size();
    void Clear();

    T Front();
    T Back();

    void Enqueue(T newData);
    void Dequeue();

    queueArr(size_t cap);
    ~queueArr();

    queueArr(const queueArr& other);              // copy constructor
    queueArr& operator=(const queueArr& other);    // copy assignment
};

// Build a new, empty queue that can hold "cap" items.
template <typename T>
queueArr<T>::queueArr(size_t cap) {
    capacity = cap;
    count = 0;
    frontIndex = 0;
    backIndex = 0;
    data = new T[capacity];
}
```

```cpp
// Free the array when the queue is destroyed.
template <typename T>
queueArr<T>::~queueArr() {
    delete[] data;
}

// Make a brand-new queue that is a copy of "other".
template <typename T>
queueArr<T>::queueArr(const queueArr& other) {
    capacity = other.capacity;
    frontIndex = other.frontIndex;
    backIndex = other.backIndex;
    count = other.count;

    data = new T[capacity];
    for (size_t i = 0; i < capacity; i++) {
        data[i] = other.data[i];
    }
}

// Copy the contents of "other" into a queue that already exists.
template <typename T>
queueArr<T>& queueArr<T>::operator=(const queueArr& other) {
    if (this == &other) {
        return *this;
    }

    delete[] data;   // free our old array first

    capacity = other.capacity;
    frontIndex = other.frontIndex;
    backIndex = other.backIndex;
    count = other.count;

    data = new T[capacity];
    for (size_t i = 0; i < capacity; i++) {
```

```
        data[i] = other.data[i];
    }

    return *this;
}

// Is the queue empty?
template <typename T>
bool queueArr<T>::Empty() {
    return count == 0;
}

// Is the queue full?
template <typename T>
bool queueArr<T>::Full() {
    return count == capacity;
}

// How many items are stored rn?
template <typename T>
size_t queueArr<T>::Size() {
    return count;
}

// Reset the queue back to empty.
template <typename T>
void queueArr<T>::Clear() {
    frontIndex = 0;
    backIndex = 0;
    count = 0;
}

// front of the queue.
template <typename T>
T queueArr<T>::Front() {
    if (Empty()) {
        throw std::runtime_error("Front() called on empty queue");
```

```
    }
    return data[frontIndex];
}

// back of the queue.
template <typename T>
T queueArr<T>::Back() {
    if (Empty()) {
        throw std::runtime_error("Back() called on empty queue");
    }
    return data[backIndex];
}

// Add new item to the back of the queue.
template <typename T>
void queueArr<T>::Enqueue(T newItem) {
    if (Full()) {
        throw std::runtime_error("Enqueue() called on full queue");
    }

    if (count == 0) {
        backIndex = 0;   // this is the very first item
    } else {
        // move one step forward, wrapping back to 0 after the last
slot
        backIndex = (backIndex + 1) % capacity;
    }

    data[backIndex] = newItem;
    count++;
}

// Remove item at the front of the queue.
template <typename T>
void queueArr<T>::Dequeue() {
    if (Empty()) {
        throw std::runtime_error("Dequeue() called on empty queue");
```

```
    }
    frontIndex = (frontIndex + 1) % capacity;    // wrap around if
needed
    count--;
}

#endif
```

OUTPUT

```
Enqueue 10, 20, 30, 40, 50

Front: 10
Back : 50
Size : 5
Full?: true

Dequeue twice
Front now: 30
Size: 3

Enqueue 60, 70
Back now: 70
Full?: true

Copy constructor:

copy.Front(): 30
copy.Back(): 70

Clear()
Empty? true
```

Code analysis:

This code creates a fixed-size circular queue using a standard array. It uses index wrapping to efficiently re-use array slots as items are added and removed.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using queue implementation

Source file

```
#include <iostream>

class Job {
public:
    int id;
    std::string u;
    int p;

    Job() {
        id = 0;
        u = "";
        p = 0;
    }

    Job(int a, std::string b, int c) {
        id = a;
        u = b;
        p = c;
    }
};

struct JobNode {
    Job data;
    JobNode* next;

    JobNode(Job j) : data(j), next(nullptr) {}
};
```

```
class JobQueue {
public:
    JobNode* f;
    JobNode* b;
    int c;

    JobQueue() {
        f = nullptr;
        b = nullptr;
        c = 0;
    }

    bool isEmpty() {
        if (f == nullptr) return true;
        else return false;
    }

    int size() {
        return c;
    }

    void enqueue(Job j) {
        JobNode* n = new JobNode(j);
        if (f == nullptr) {
            f = n;
            b = n;
        } else {
            b->next = n;
            b = n;
        }
        c = c + 1;
    }

    bool dequeue(Job &o) {
        if (f == nullptr) return false;
        JobNode* t = f;
        o = t->data;
```

```
        f = f->next;
        if (f == nullptr) b = nullptr;
        delete t;
        c = c - 1;
        return true;
    }

    ~JobQueue() {
        Job x;
        while (dequeue(x)) {}
    }
};

class Printer {
public:
    JobQueue q;
    int nid;

    Printer() {
        nid = 1;
    }

    void addJob(std::string u, int p) {
        Job j(nid, u, p);
        nid++;
        q.enqueue(j);
        std::cout << "Added job #" << j.id << " for " << u << " (" << p
<< " pages) to the list." << std::endl;
    }

    void processAllJobs() {
        std::cout << std::endl << "Printer starting! Total jobs to
print: " << q.size() << std::endl;
        Job cur;
        while (q.dequeue(cur)) {
            std::cout << "Printing job #" << cur.id << " for " << cur.u
<< " (" << cur.p << " pages)... Done!" << std::endl;
```

```
        }
        std::cout << "All done! No jobs left." << std::endl;
    }

    bool hasPendingJobs() {
        if (!q.isEmpty()) return true;
        else return false;
    }
};

int main() {
    Printer p;

    std::cout << "People are adding print jobs" << std::endl;
    p.addJob("User 1", 3);
    p.addJob("User 2", 12);
    p.addJob("User 3", 1);
    p.addJob("User 4", 7);

    p.processAllJobs();

    std::cout << std::endl << "New jobs came in!" << std::endl;
    p.addJob("User 6", 5);
    p.addJob("User 7", 2);

    p.processAllJobs();

    return 0;
}
```

OUTPUT

```
People are adding print jobs
Added job #1 for User 1 (3 pages) to the list.
Added job #2 for User 2 (12 pages) to the list.
Added job #3 for User 3 (1 pages) to the list.
Added job #4 for User 4 (7 pages) to the list.

Printer starting! Total jobs to print: 4
Printing job #1 for User 1 (3 pages)... Done!
Printing job #2 for User 2 (12 pages)... Done!
Printing job #3 for User 3 (1 pages)... Done!
Printing job #4 for User 4 (7 pages)... Done!
All done! No jobs left.

New jobs came in!
Added job #5 for User 6 (5 pages) to the list.
Added job #6 for User 7 (2 pages) to the list.

Printer starting! Total jobs to print: 2
Printing job #5 for User 6 (5 pages)... Done!
Printing job #6 for User 7 (2 pages)... Done!
All done! No jobs left.
```

Code analysis:

This code simulates a real-world print job manager using a linked-list queue. It processes document requests one by one in the exact order they were received.

Conclusion:

Queues are fundamental first-in, first-out **(FIFO)** data structures that can be created using C++ STL, custom linked lists, or circular arrays. Learning these different approaches demonstrates how queues efficiently organize sequential tasks, such as real-world printer job management.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic

COMPUTER ENGINEERING

QUEZON CITY

work. The following indicates the nature and extent of AI assistance:

Tools Used: Claude

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: *Idea generation, formatting, summarization, code assistance, and explanation of cncepts*

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: I used AI as a helper to make my explanations easy to read. I wrote the code logic, tested it .

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.